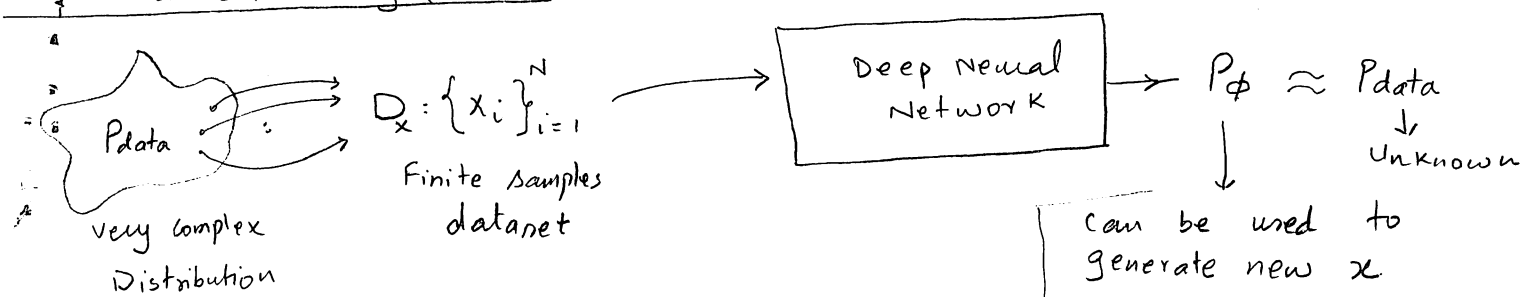


Deep Generative Modeling (DGM):



Training Deep network requires to minimize some discrepancy/divergence as:

$$\phi^* = \arg \min_{\phi} D(P_{data}, P_\phi)$$

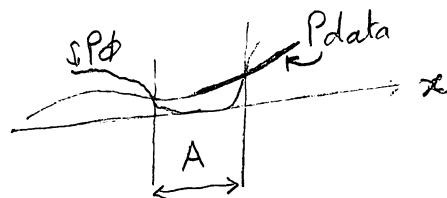
$D(\cdot, \cdot)$ = Some divergence measure $\rightarrow$ we must be able to compute it using $D_x: \{x_i\}_{i=1}^N$

① KL-Divergence:

$$D_{KL}(P_{data} \parallel P_\phi) = E_{x \sim P_{data}} \left[\log(P_{data}(x)) - \log(P_\phi(x)) \right] = \int P_{data}(x) \log \frac{P_{data}(x)}{P_\phi(x)} dx$$

Forward KL

$\downarrow$
It is mode covering: in region A,



$P_{data}(A) > 0$ but model assigns

$$P_\phi(x) = 0 \Rightarrow \log \frac{P_{data}(x)}{0} = +\infty \Rightarrow D_{KL} = \infty \Rightarrow \text{minimizing the } D_{KL}$$

will push the model to put probability mass here.

Since, $D_{KL}(P_{data} \parallel P_\phi)$ is not computable as $P_{data}(x)$ is not available;

So, we write it as:

$$D_{KL}(\cdot) = E_{P_{data}} [\log P_{data}(x)] - E_{P_{data}} [\log P_\phi(x)]$$

$$= \text{Entropy} = H(P_{data})$$

$$\Rightarrow D_{KL}(\cdot) = \underbrace{-E_{P_{data}} [\log P_\phi(x)]}_{\text{Cross-entropy blw } P_{data} \text{ \& } P_\phi} + \underbrace{H(P_{data})}_{\text{Entropy of } P_{data}} \rightarrow \text{①}$$

does not depend on ϕ

Minimizing ① is equivalent to maximizing $-E_{P_{data}} [\log P_\phi(x)]$

$$\min_{\phi} D_{KL}(P_{data} \parallel P_{\phi}) \iff \max_{\phi} E_{x \sim P_{data}} \left[\log P_{\phi}(x) \right] \rightarrow \text{Maximize LLH of data to find } \phi: \text{ Those } x \text{ which resemble images, get high prob.}$$

Replace $E[\cdot]$ with empirical average as: $\mathcal{L}_{MLE}(\phi) = -\frac{1}{N} \sum_{i=1}^N \log P_{\phi}(x_i)$

② Fisher-Divergence:

For distributions P & Q , we define: (e.g. $P = P_{data}$ & $Q = P_{\phi}$)

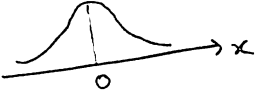
$$D_F(P \parallel Q) = E_{x \sim P} \left[\left\| \nabla_x \log P(x) - \nabla_x \log Q(x) \right\|_2^2 \right] \rightarrow \textcircled{2}$$

where $\nabla_x \log P(x)$ = Score function of $P(x)$

& $\nabla_x \log Q(x)$ = " " " " $Q(x)$

Score function = ∇ of log-density w.r.t data-variable x . It is a local direction field telling us: "If we move x a tiny bit, in which direction, does the log-prob. increase the fastest?"

Ex. $P(x) = \mathcal{N}(0, \sigma^2)$



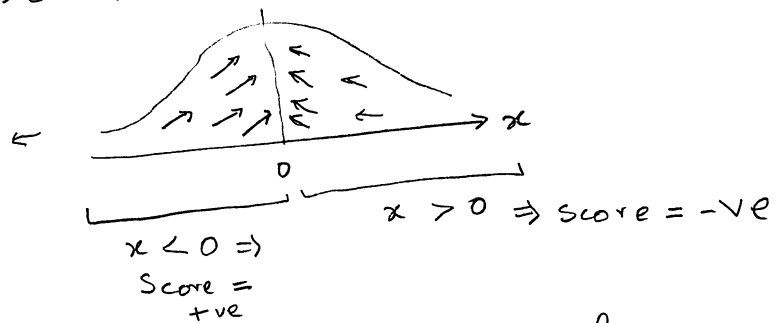
$$\log P(x) = -\frac{x^2}{2\sigma^2} + C$$

$$\nabla_x \log P(x) = -\frac{x}{\sigma^2} \rightarrow \text{if } x < 0 \Rightarrow \text{Score} = +ve \rightarrow \text{it points right (towards 0)}$$

$$\rightarrow \text{if } x > 0 \Rightarrow \text{Score} = -ve \rightarrow \text{it points left (towards 0)}$$

Magnitude changes as $\left| \frac{x}{\sigma^2} \right|$

as we move away from high-density region ($x=0$)



Score = Points "uphill" towards higher density like a force pulling samples towards modes.

Why ② makes sense? Matching scores matches their distributions (upto a const.)

$$\text{If } \nabla_x \log P(x) = \nabla_x \log Q(x)$$

$$\Rightarrow \nabla_x \log P(x) - \nabla_x \log Q(x) = 0$$

$$\Rightarrow \log P(x) - \log Q(x) = \underbrace{\text{const.}}_{=C}$$

$$\Rightarrow \boxed{P(x) = C Q(x)} \rightarrow \text{If } P \text{ \& } Q \text{ are normalized densities i.e. } \int P = \int Q = 1 \Rightarrow C=1$$

challenges in modeling $P_\phi(x)$:



②

$P_\phi(x)$ must satisfy:

① $P_\phi(x) \geq 0$ ② $\int P_\phi(x) dx = 1$

However, NN can output any real value $E_\phi(x)$ as network output.

A common way to deal with this could be:

→ Enforce positivity: $\tilde{P}_\phi(x) = e^{E_\phi(x)}$

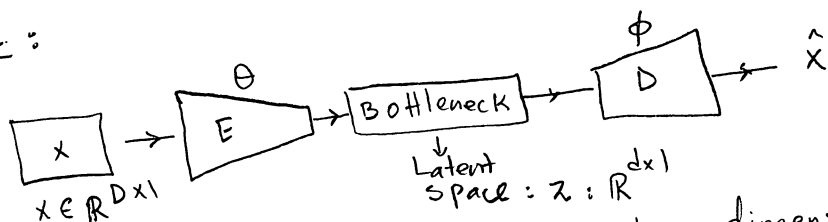
→ Normalize: $P_\phi(x) = \frac{e^{E_\phi(x)}}{\int \tilde{P}_\phi(x') dx'} = \frac{e^{E_\phi(x)}}{\int e^{E_\phi(x')} dx'} = \frac{e^{E_\phi(x)}}{Z(\phi)}$

$Z(\phi)$ = Normalizing const. / Partition factor.

In high-dimensions (e.g. for text, audio, video, images), computing $Z(\phi)$ is a main bottleneck. All generative models work either:

- by avoiding $Z(\phi)$
 - " approximating $Z(\phi)$
 - by circumventing this bottleneck
- $\left\{ \begin{array}{l} \textcircled{1} \text{ Energy Based Models (EBMs)} \\ \textcircled{2} \text{ AR models} \\ \textcircled{3} \text{ VAEs} \\ \textcircled{4} \text{ Normalizing Flows (NFs) / Diffusion} \\ \textcircled{5} \text{ GANs} \end{array} \right.$

Autoencoder:



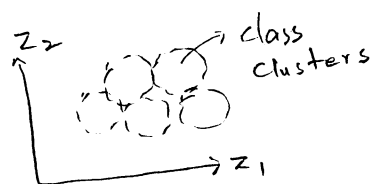
E : compresses data into z which is low-dimensional-space

D : Reconstructs x from z

Training: $L_{\theta, \phi}(x, \hat{x}) = \frac{1}{N} \sum_{i=1}^N (x_i - \hat{x}_i)^2$



Let $d=2$:
 $z: \mathbb{R}^{2 \times 1}$



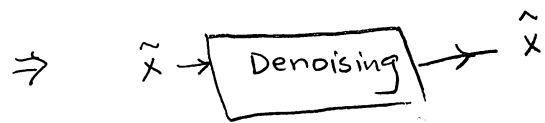
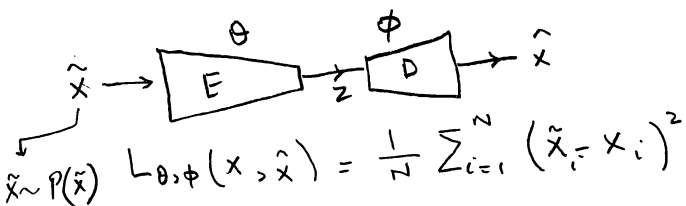
d = latent dimension = most imp. design choice.
 d = too small $\rightarrow$ bad reconstruction.
 d = too large $\rightarrow$ compression goes away

Limitations: • clusters are not well-defined.

- Interpolation b/w any two points in z_1 - z_2 plane, produces no meaningful $\hat{x}$.
- Add noise to x & everything fails.

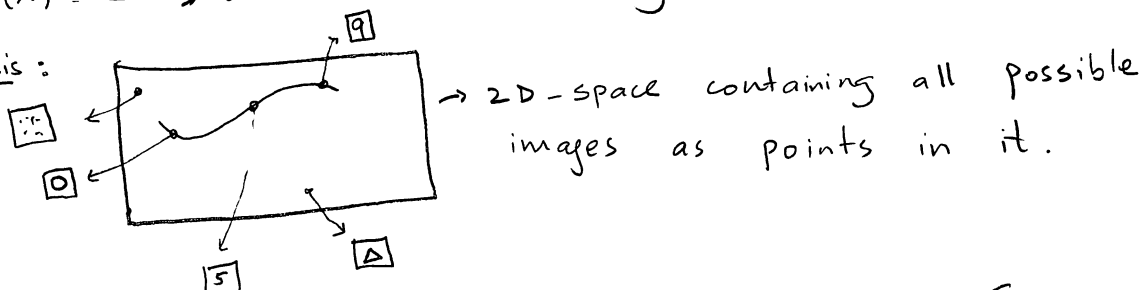
solution: • Regularize AE

Denoising AE: x = Clean Image, $\tilde{x} = x + \epsilon$ = Noisy Image.
 $\epsilon \sim \mathcal{N}(0, \sigma)$: AWGN, σ controls the amount

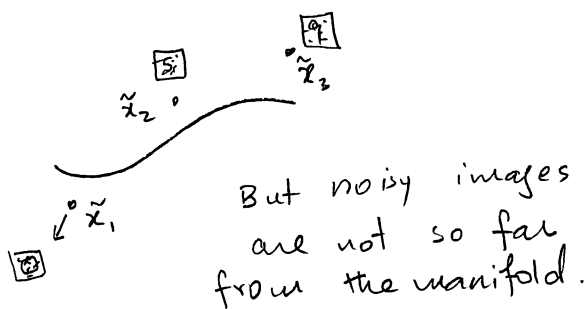
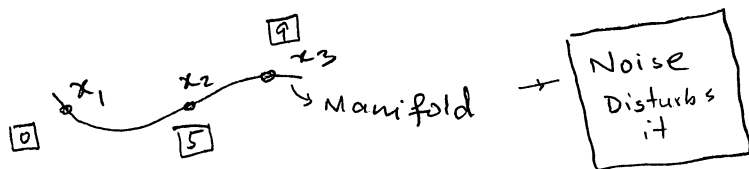


$\hookrightarrow f(\tilde{x})$ what is $f(\cdot)$ learning?

Manifold Hypothesis:



Key idea: Meaningful images contain only a tiny fraction of this space.
 (Among all possible pixel arrangements, only a subset correspond to coherent & readable arrangements)



- $f(\cdot)$ learns a transformation which maps these noisy images back on data manifold.
 By learning shapes & structures of images.

Mean of Posterior Pdf

$$\hat{x} = \arg \min_f E[(x - f(\tilde{x}))^2]$$

$$\Rightarrow f(\cdot) \approx E[x | \tilde{x}] =$$

Posterior mean:

$$E_{P(x|\tilde{x})}[x]$$

Posterior Pdf

$$f(\tilde{x}) = E[x | \tilde{x}]$$

$$\tilde{x} + \sigma \nabla \log P(\tilde{x})$$

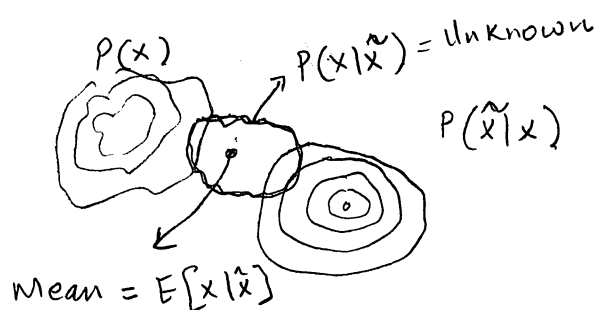
Tweedie formula

input to NN

Apply a small gradient step in the direction of score of data distribution

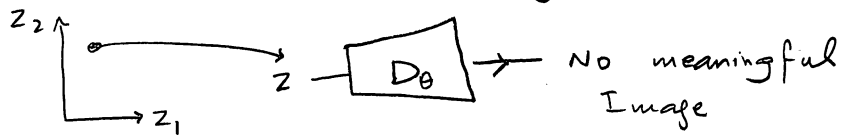
$\Rightarrow$ Move data to regions having higher density

$\hookrightarrow$ Manifold Hypothesis



VAE's: Latent space of simple AE is disorganized & irregular; even

very small movements from points, result in loss of structure:



Structure: $\min_{\theta, \phi} E_{x \sim P_{data}} [\|x - D_{\theta}(z)\|_2^2]$ with $z = E_{\phi}[x]$

$M = \{E_{\phi}(x) : x \in \text{Training Data}\}$

Decoder only learns to decode (reconstruct) this set effectively.

If $M = \text{Disconnected/Thin}$
Twisted set $\Rightarrow$ a random latent z (when sampled) $\notin M$
 $\Rightarrow D_{\theta}(z) = \text{Garbage}$.

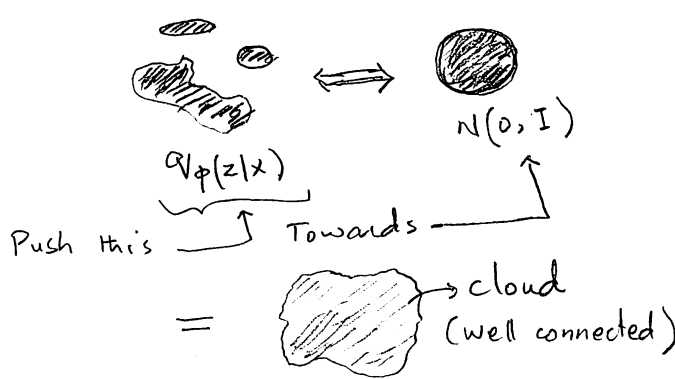
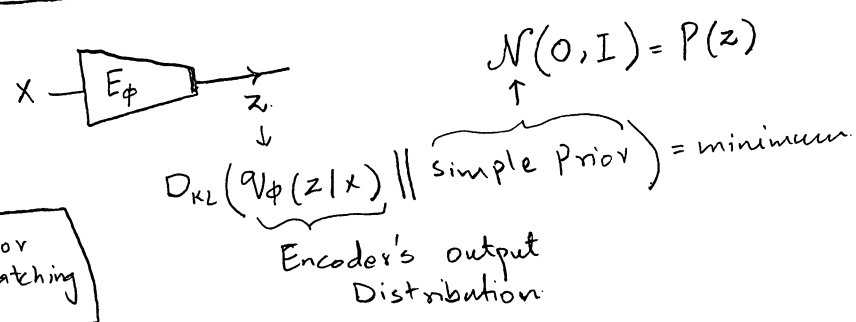
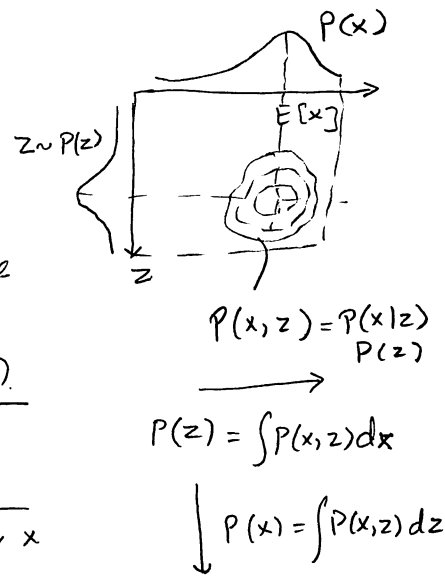
VAE changes this by forcing z to resemble like a simple well-connected prior:

$$P(x|z) = \frac{P(x, z)}{P(z)}$$

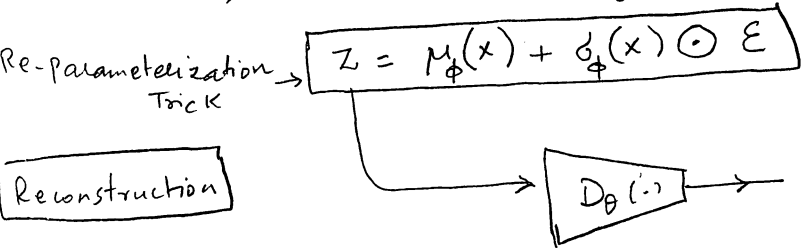
= Joint Prob. of sampling value of z

$$P(z|x) = \frac{P(x, z)}{P(x)}$$

= " " " x



Further, in order to enforce nearby z 's produce similar x 's, it uses noise injection in latent:



where $\epsilon = \text{Noise}$
 $\mu_{\phi} = \text{Mean of } q_{\phi}(z|x)$
 $\sigma_{\phi} = \text{std. " "}$

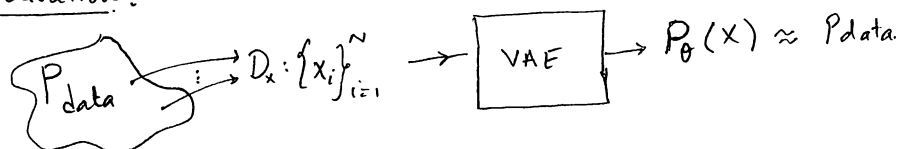
$$L_{ELBO}(\phi, \theta) = -E_{q_{\phi}} [\log P_{\theta}(x|z)] + D_{KL} \left(q_{\phi}(z|x) \parallel P_{\theta}(z) \right)$$

$L_{ELBO}(\phi, \theta) = -ELBO$ where $ELBO = E_{q_{\phi}}[\cdot] - D_{KL}(\cdot \parallel \cdot)$

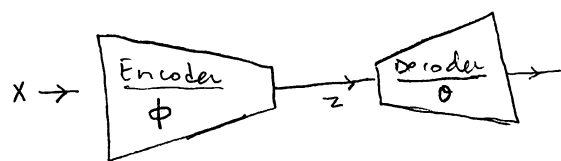
$E_{q_{\phi}}[\cdot]$ is evaluated by sampling $z \sim q_{\phi}(z|x)$:

$$E_{q_{\phi}}[\cdot] \approx \frac{1}{N} \sum_{i=1}^N \log P_{\theta}(x|z^{(i)})$$

VAE Derivation:



Use a latent-variable z & split Encoder-Decoder:



Let $z \sim q_\phi(z|x)$ & assume $P_\theta(x) = \int P_\theta(x|z) P_\theta(z) dz$
 $\approx q_\phi(x|z) = \mathcal{N}(0, I)$

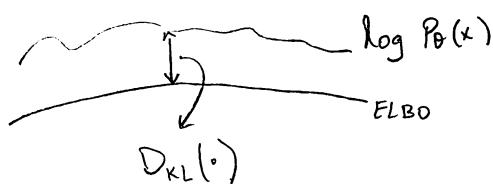
$$\max_{\theta} E_{q_\phi} [\log P_\theta(x)] = \max_{\theta} E_{q_\phi} \left[\log \frac{P_\theta(x, z)}{P_\theta(z|x)} \right]$$

$$\therefore P_\theta(z|x) = \frac{P_\theta(x, z)}{P_\theta(x)}$$

$$= E_{q_\phi} \left[\log \frac{P_\theta(x|z)}{P_\theta(z|x)} \cdot \frac{q_\phi(z|x)}{q_\phi(z|x)} \right] = \underbrace{E_{q_\phi} \left[\log \frac{P_\theta(x|z)}{q_\phi(z|x)} \right]}_{\text{ELBO Term}} + \underbrace{E_{q_\phi} \left[\log \frac{q_\phi(z|x)}{P_\theta(z|x)} \right]}_{D_{KL}(q_\phi(z|x) || P_\theta(z|x))}$$

$$\Rightarrow \boxed{\log P_\theta(x) \geq \text{ELBO Term}}$$

ELBO = lower bound on LLH ($\because D_{KL} \geq 0$)



$$\Rightarrow \max \log P_\theta(x) \Rightarrow \max E_{q_\phi} \left[\frac{P_\theta(x, z)}{q_\phi(z|x)} \right]$$

$$\Rightarrow \max_{\theta, \phi} E_{q_\phi} \left[\frac{P_\theta(x, z)}{q_\phi(z|x)} \right] = \max_{\theta, \phi} E_{q_\phi} [P_\theta(x|z)] - \underbrace{D_{KL}(q_\phi(z|x) || P_\theta(z))}_{\substack{\text{Posterior} \\ (\text{Approx})} \quad \text{Prior}}$$

$$L_{\text{ELBO}}(\theta, \phi) = -\text{ELBO} = \underbrace{D_{KL}(\cdot || \cdot)}_{\text{Encoder}} - \underbrace{E_{q_\phi} [P_\theta(x|z)]}_{\text{Decoder}}$$

↑ minimize it

$$\nabla_{\theta} L_{\text{ELBO}} = -E_{q_\phi} [\nabla_{\theta} \log P_\theta(x|z)] \approx \nabla_{\theta} \log P_\theta(x|z^{(i)}) \quad \text{where } z^{(i)} \sim q_\phi(z|x)$$

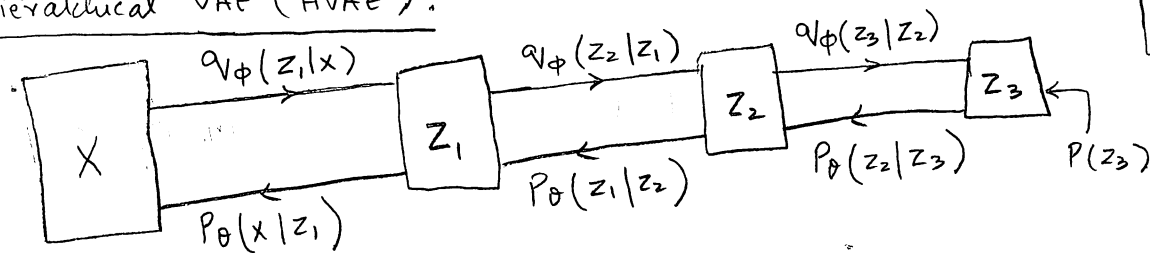
$$\nabla_{\phi} L_{\text{ELBO}} = \nabla_{\phi} E_{q_\phi} [\log P_\theta(x|z) - \log q_\phi(z|x)] \xrightarrow{f_1(z)}$$

Not possible

The re-parameterization trick: $z = \mu + \sigma \odot \epsilon$ & $\epsilon \sim \mathcal{N}(0, I)$

$$\nabla_{\phi} L_{\text{ELBO}} = E_{\epsilon \sim \mathcal{N}(0, I)} [\nabla_{\phi} (\quad)] \xrightarrow{f_1(z)}$$

Hierarchical VAE (HVAE):



$q_\phi(z_1, z_2, z_3 | x) = q_\phi(z_1 | x) q_\phi(z_2 | z_1) q_\phi(z_3 | z_2)$ ← Markovian assumption allows us to factorize over all latents.

$p_\theta(x, z_1, z_2, z_3) = p(z_3) p_\theta(z_2 | z_3) p_\theta(z_1 | z_2) p_\theta(x | z_1)$ ← Defines our generative direction ($z_3 \rightarrow z_2 \rightarrow z_1 \rightarrow x$)

Inference Model:

$$\begin{aligned} q_\phi(z_1 | x) &\sim \mathcal{N}(\mu_{\phi_1}, \Sigma_{\phi_1}) \\ q_\phi(z_2 | z_1) &\sim \mathcal{N}(\mu_{\phi_2}, \Sigma_{\phi_2}) \\ q_\phi(z_3 | z_2) &\sim \mathcal{N}(\mu_{\phi_3}, \Sigma_{\phi_3}) \end{aligned}$$

with $\Sigma_{\phi_l} = \text{diag}(\sigma_{\phi_l}^2)$

Reparameterization

$$\begin{aligned} z_l &= \mu_{\phi_l} + \sigma_{\phi_l} \odot \epsilon_l \\ \epsilon_l &\sim \mathcal{N}(0, I) \end{aligned}$$

Generative Model:

$$\begin{aligned} p_\theta(z_3) &\sim \mathcal{N}(0, I) \\ p_\theta(z_2 | z_3) &\sim \mathcal{N}(\mu_{\theta_2}, \Sigma_{\theta_2}) \quad \Sigma_{\theta_l} = \text{diag}(\sigma_{\theta_l}^2) \\ p_\theta(z_1 | z_2) &\sim \mathcal{N}(\mu_{\theta_1}, \Sigma_{\theta_1}) \\ p_\theta(x | z_1) &\sim \mathcal{N}(\mu_{\theta_x}, \sigma_x^2 I) \end{aligned}$$

$$\begin{aligned} \mathcal{L}_{ELBO}(\phi, \theta) = & - \mathbb{E}_{q_\phi(z_1, z_2, z_3)} \left[\log p_\theta(x | z_1) \right] + \mathbb{E}_{q_\phi(z_1 | x)} \left[D_{KL}(q_\phi(z_1 | x) \| p_\theta(z_1 | z_2)) \right] \\ & + \mathbb{E}_{q_\phi(z_2 | z_1)} \left[D_{KL}(q_\phi(z_2 | z_1) \| p_\theta(z_2 | z_3)) \right] + D_{KL}(q_\phi(z_3 | z_2) \| p_\theta(z_3)) \end{aligned}$$

Training aligns each inference-conditional to its corresponding generative conditional.

Benefits: ① More expressive prior: Vanilla VAE: $p_\theta(z) \sim \mathcal{N}(0, I)$
HVAE: $p_\theta(z_1) = \int p_\theta(z_1 | z_2) p_\theta(z_2 | z_3) p_\theta(z_3) dz_2 dz_3$
Highly non-Gaussian & multi-modal.

② The chain ($z_3 \rightarrow z_2 \rightarrow z_1 \rightarrow x$) contains distributed information.

Details Global factors

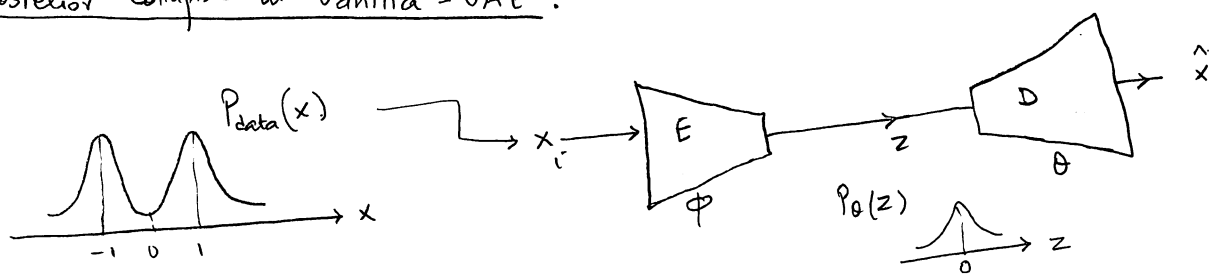
Cons: ① still blurriness: $p_\theta(x | z_1) \sim \mathcal{N}(\mu_{\theta_x}, \sigma_x^2 I)$

② Posterior Collapse: happens when $q_\phi(z | x) \approx p_\theta(z)$ → Latent z carries no information about x
↳ is present in both Vanilla-VAE & HVAE

③ Training = harder

It happens when Decoder is very strong e.g. multi-modal AR decoder

Posterior Collapse in Vanilla-VAE :



$$L_{ELBO}(\theta, \phi) = - \mathbb{E}_{q_{\phi}(z|x)} [\log P_{\theta}(x|z)] + D_{KL}(q_{\phi}(z|x) \parallel P_{\theta}(z))$$

If Decoder is strong, it can directly model $P_{data}(x)$:

$$P_{\theta}(x|z) = P_{data}(x) \quad \forall x_i$$

$$\Rightarrow \mathbb{E}_{q_{\phi}(z|x)} [\log P_{\theta}(x|z)] = \log P_{\theta}(x|z) = \log P_{data}(x)$$

$$L_{ELBO}(\theta, \phi) = \underbrace{- \log P_{data}(x)}_{\text{fixed wrt } \theta \& \phi} + D_{KL}(q_{\phi}(z|x) \parallel P_{\theta}(z))$$

Reducing $L_{ELBO}(\theta, \phi)$ will only decrease $D_{KL}(\cdot \parallel \cdot) \Rightarrow$

$$D_{KL}(\cdot \parallel \cdot) = 0 \quad \text{when} \quad q_{\phi}(z|x) = P_{\theta}(z) \Rightarrow \text{Approximate}$$

Posterior $q_{\phi}(z|x)$ (at this point) contains no information about x at all.

$\rightarrow z = \text{latent}$ was a control $\rightarrow$ changing z gave us different x in controllable way. Under posterior-collapse, $z = \text{useless}$

VAE works but produces blurry images :

- 1- Latent is bottleneck $\rightarrow$ reduces a lot of information which are not then recoverable
- 2- $P_\theta(x|z) \sim \mathcal{N}(\mu_\theta, \Sigma_\theta)$ = Gaussian but if actual $P_\theta(x|z) = \text{wavy line}$ $\rightarrow$ blurry images.

Interpolation in VAE latent is meaningful : because we forced $q_\phi(z|x)$ to resemble like $\mathcal{N}(0, I)$ $\rightarrow$ which forces latent space to be filled/smooth.

VAE variants :

① Vanilla VAE : assumes $P_\theta(x|z) \sim \mathcal{N}(\mu_\theta, \Sigma_\theta)$ with $\Sigma_\theta = \sigma_\theta^2 I$ & $\sigma_\theta^2 = \text{fixed}$

This leads to $-\log P_\theta(x|z) = \frac{1}{2\sigma_\theta^2} \|x - \mu_\theta(z)\|_2^2 + \text{const.}$

② Vanilla VAE with learned σ^2 : $\Sigma_\theta = \text{diag}(\sigma_i^2)$

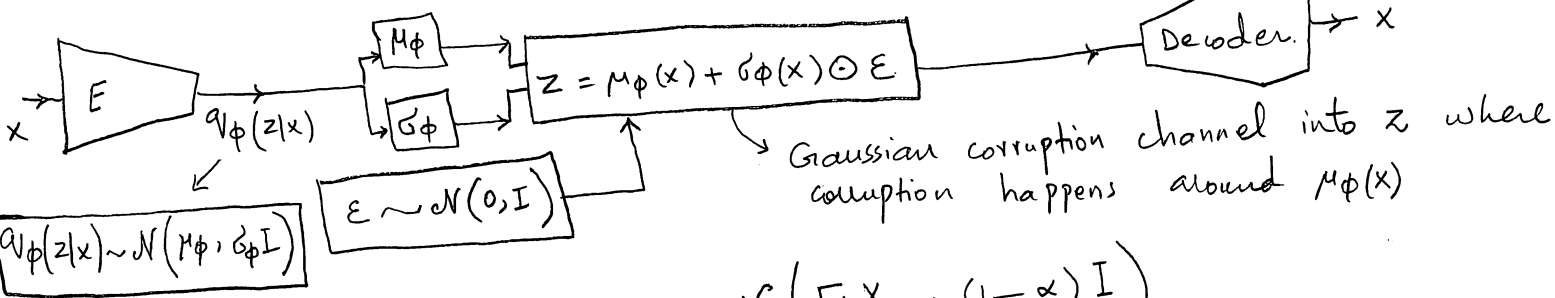
$\Rightarrow -\log P_\theta(x|z) = \frac{1}{2} \sum_i \left[\frac{(x_i - \mu_i)^2}{\sigma_i^2} + \log \sigma_i^2 \right] + \text{const.}$

③ Bernoulli LL-Assumption : $P_\theta(x|z) = \prod_i \text{Bernoulli}(x_i, \hat{x}_i)$ $\hat{x}_i = \sigma(\text{logits}(z))$
 $\hat{x}_i \in \{0, 1\}$

$\Rightarrow -\log P_\theta(x|z) = \sum_i [-x_i \log \hat{x}_i - (1-x_i) \log (1-\hat{x}_i)]$

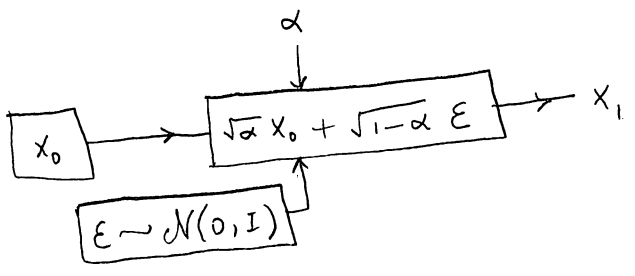
VAE without Encoder : A bridge to Diffusion : standard VAE has two coupled issues: ① Learn encoder $q_\phi(z|x)$ ② Learn decoder $P_\theta(x|z)$

Let us remove ① by sampling from only Gaussian & focus only on denoising/reconstruction



Let us fix E as : $q(x_1|x_0) \sim \mathcal{N}(\underbrace{\sqrt{\alpha} x_0}_{\text{mean}}, \underbrace{(1-\alpha) I}_{\text{std.}})$

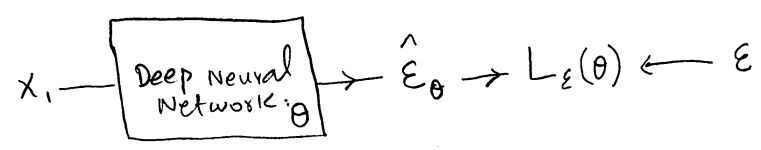
Now, reparameterization becomes : $x_1 = \sqrt{\alpha} x_0 + \sqrt{1-\alpha} \epsilon$
Mean & std. are fixed & value of α controls them. (consider $x_1 \equiv z$, $x_0 \equiv x$)



How to go back to x_0 = ?

$x_1 = \sqrt{\alpha} x_0 + \sqrt{1-\alpha} \epsilon$
 $\Rightarrow x_0 = \frac{x_1 - \sqrt{1-\alpha} \epsilon}{\sqrt{\alpha}}$

Given only x_1 , we can train a network to predict ϵ & then use this eq. to recover x_0 .



$$L_{\epsilon}(\theta) = E_{x_0, \epsilon} \left[\left\| \epsilon - \epsilon_{\theta}(x_1) \right\|_2^2 \right] \rightarrow \min_{\theta} L_{\epsilon}(\theta)$$

Once we have ϵ_{θ} , we can approximate x_0 as:

$$\hat{x}_0 = \frac{x_1 - \sqrt{1-\alpha} \epsilon}{\sqrt{\alpha}} \Rightarrow \left\| \hat{x}_0 - x_0 \right\|_2^2 = \left\| \frac{x_1 - \sqrt{1-\alpha} \epsilon}{\sqrt{\alpha}} - \frac{x_1 - \sqrt{1-\alpha} \epsilon}{\sqrt{\alpha}} \right\|_2^2$$

$$= \frac{1}{\alpha} \left\| 1 - \sqrt{\alpha} (\epsilon - \epsilon_{\theta}) \right\|_2^2$$

$$\boxed{\left\| \hat{x}_0 - x_0 \right\|_2^2 = \frac{1-\alpha}{\alpha} \left\| \epsilon - \epsilon_{\theta} \right\|_2^2}$$

Role of α :

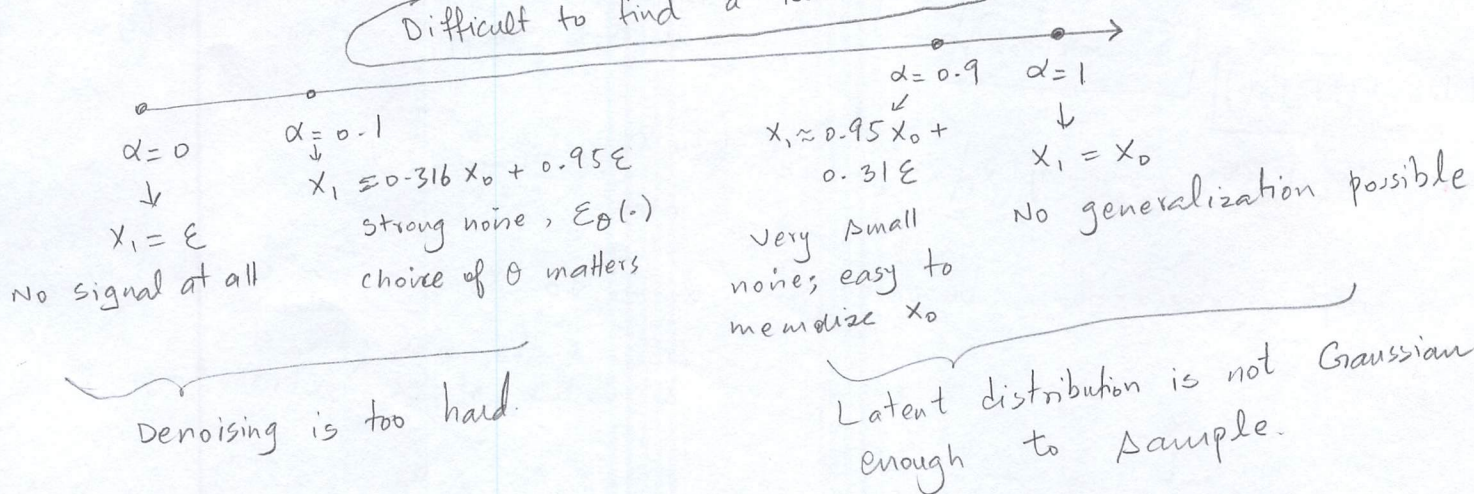
$$x_1 = \sqrt{\alpha} x_0 + \sqrt{(1-\alpha)} \epsilon \quad \left| \quad \hat{x}_0 = \frac{x_1 - \sqrt{1-\alpha} \epsilon}{\sqrt{\alpha}} \quad \right| \quad \left\| \hat{x}_0 - x_0 \right\|^2 = \left(\frac{1-\alpha}{\alpha} \right) \left\| \epsilon - \epsilon_{\theta} \right\|^2$$

$$\epsilon \sim \mathcal{N}(0, I)$$

$\alpha = \text{small}$: Matching with Gaussian prior good but little signal (x_0) remains in x_1 . No model will work.

$\alpha = \text{Large}$: More signal contribution in x_1 but large deviation from Gaussian prior. Model must predict $\epsilon_{\theta} \approx 0 \Rightarrow$ Training becomes unstable or trivial.

Difficult to find a sweet spot

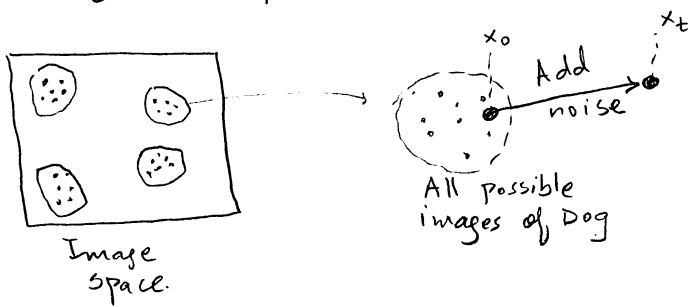


(Diffusion models provide multi-step solution to this problem.)

Here, in encoder-free setting, there is no learned inference model $q_{\phi}(z|x)$ & no "latent-to-prior KL" term. $q(x_1|x_0)$ always depends on x_0 by construction $(x_1 = \sqrt{\alpha} x_0 + \sqrt{(1-\alpha)} \epsilon) \rightarrow$ No posterior collapse

Image space: where all possible images live; most of it is empty
 ; similar images form closed clusters (A)

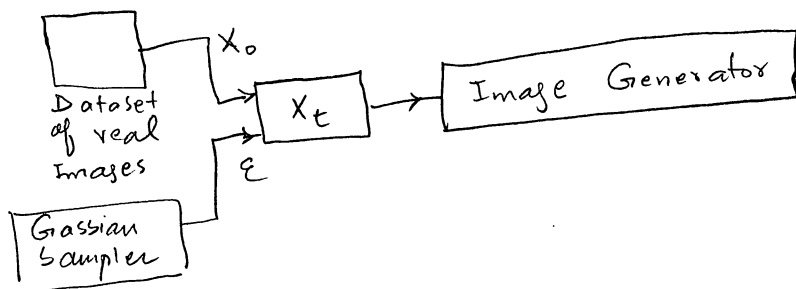
Image = sample from a pdf



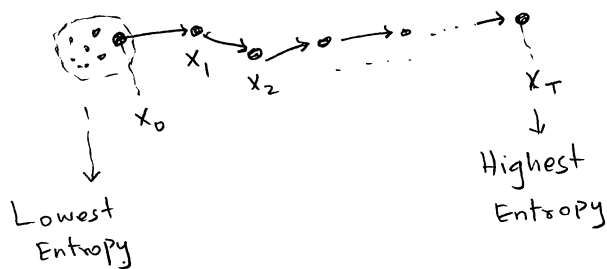
x_0, x_t, ϵ_t
 supervised learning
 pair by adding
 $\epsilon_t \sim \mathcal{N}(\cdot)$

Adding Noise
 Moving in a
 direction in
 image space.

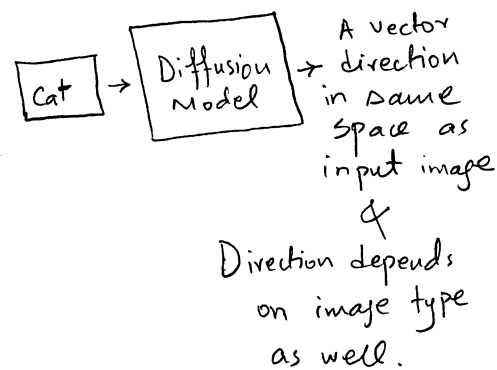
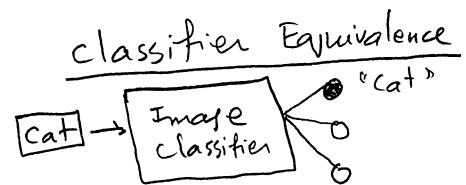
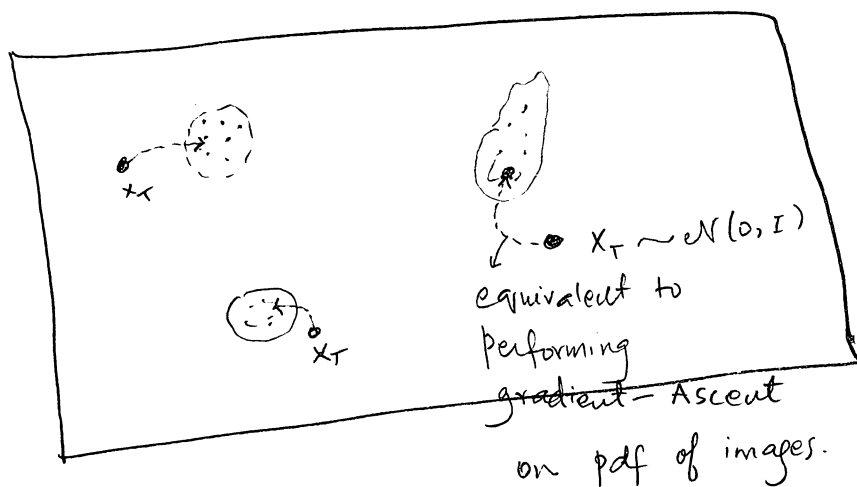
If we can get samples from a Gaussian-distribution, we get data to
 train image-generators



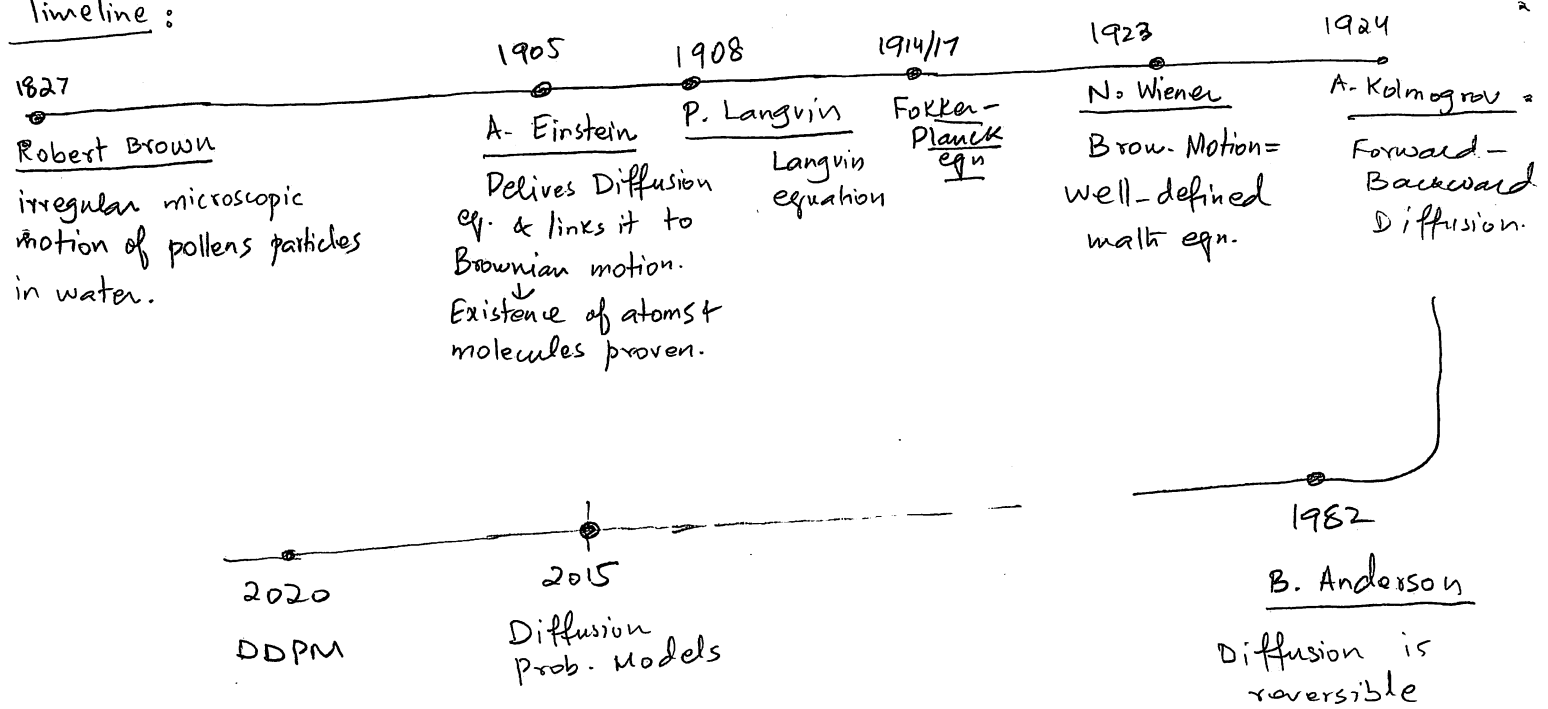
Diffusion: Take clean $x_0 \rightarrow$ Add Gaussian Noise in multiple steps.
 we move from structured cluster to empty spaces:



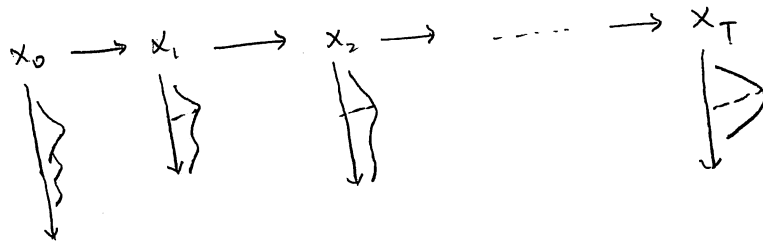
Diffusion models learn to navigate through this image space by learning
 a vector-field $\nabla_x \log P(x)$. If we sample from $\mathcal{N}(0, I)$, we are
 somewhere in empty-space $\rightarrow$ By following the direction of this
 vector field, we move to closest cluster of good images.



Timeline :



Random Process : A collection of random-variables over time.



characterized by joint :

$q(x_0, x_1, \dots, x_T) \rightarrow T$ -th order joint pdf

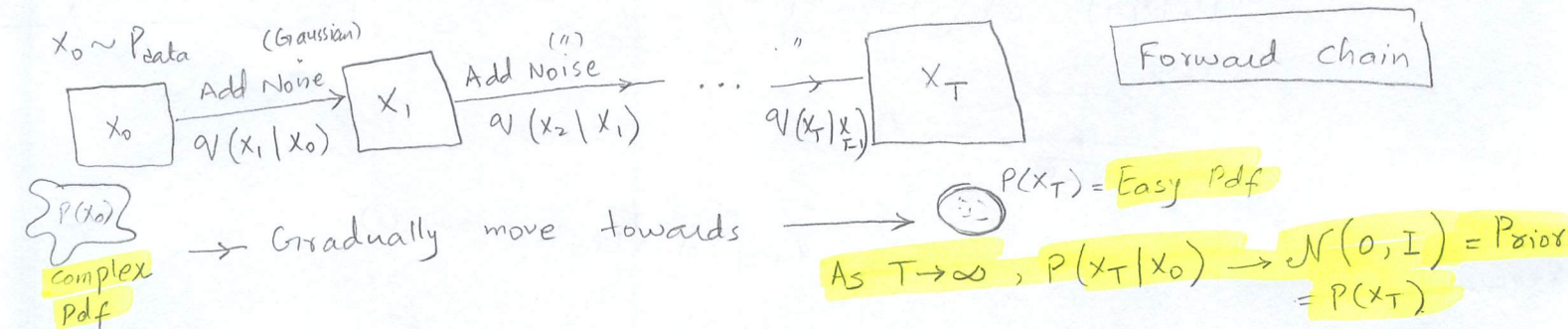
To simplify, we make some assumptions :

Markovian Assumption : $q(x_t | x_{t-1}, x_{t-2}, \dots, x_0) = q(x_t | x_{t-1})$

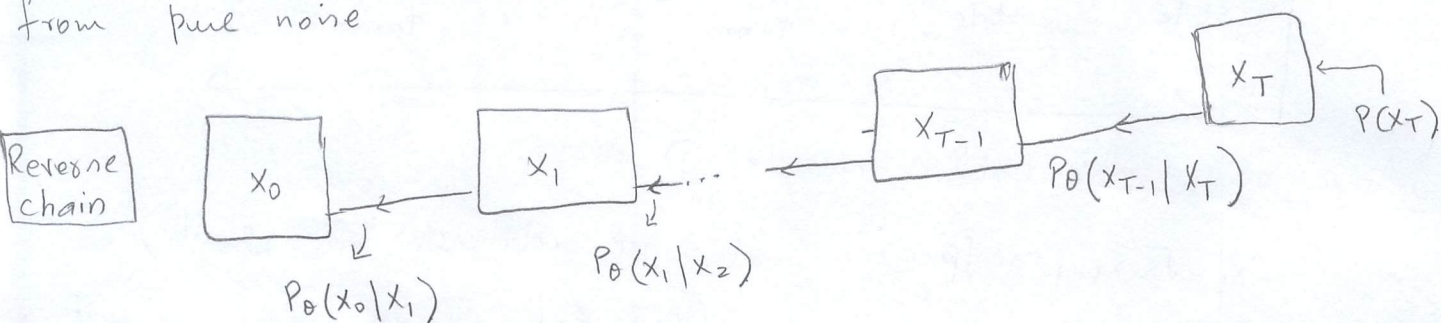
$\uparrow$
 Given present
 $\downarrow$
 Future is indep- of
 past ($x_{t-2}, x_{t-3}, \dots, x_0$)

Denoising-Diffusion Probabilistic Models (DDPM): generalizes encoder-free approach to multi-step noising-denoising process: ⑤

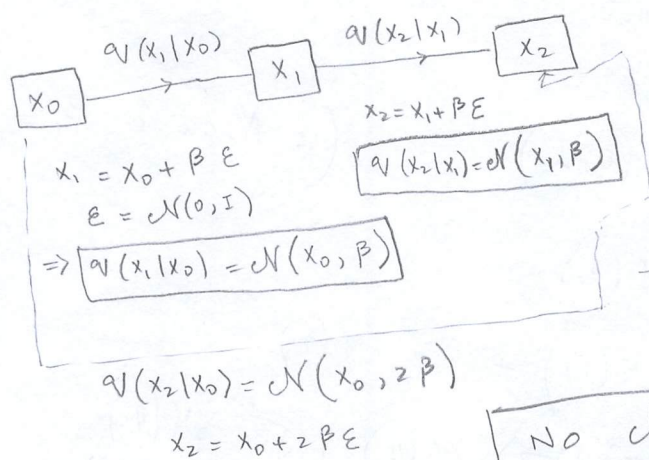
Forward Process (fixed encoder): Corrupt data x_0 by injecting Gaussian noise over multiple steps via a transition kernel $q(x_t | x_{t-1})$



Reverse Process (learnable decoder): A NN learns to reverse the noise corruption through a parameterized distribution $p_\theta(x_{t-1} | x_t)$ starting from pure noise

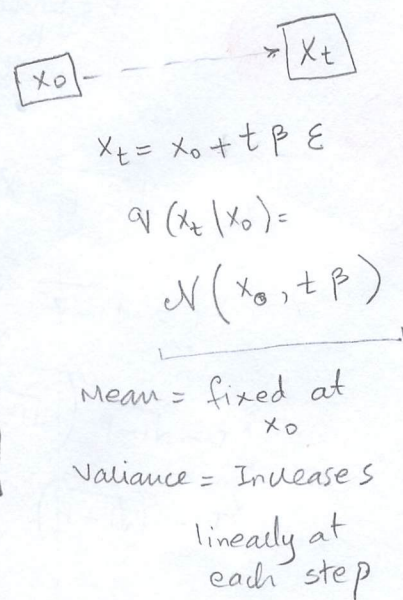


Forward Process: How to add noise? Goal: $q(x_T) = P(x_T) = q(x_T | x_0) = \mathcal{N}(0, I)$



$\rightarrow$ Repeat $\rightarrow$

No chance we reach at $\mathcal{N}(0, I)$

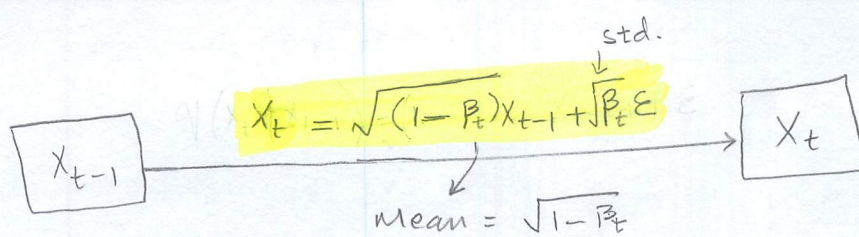


Our goal:

$$q(x_t | x_0) \xrightarrow{t \rightarrow \infty} \mathcal{N}(0, I)$$

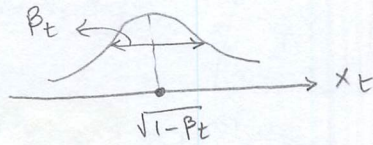
$\rightarrow$ Easy to sample + Known-fixed-data-independent Boundary condition

Variance-Exploding Diffusion



Apart from $\text{mean} = \sqrt{1-\beta}$, there could be other choices as well but this is simple

$$\mathcal{N}(X_t | X_{t-1}) = \mathcal{N}\left(\sqrt{1-\beta_t}, \underbrace{\beta_t I}_{\text{variance}}\right)$$



Reparameterization trick: $X_t = \sqrt{1-\beta_t} X_{t-1} + \sqrt{\beta_t} \epsilon \rightarrow \textcircled{1}$

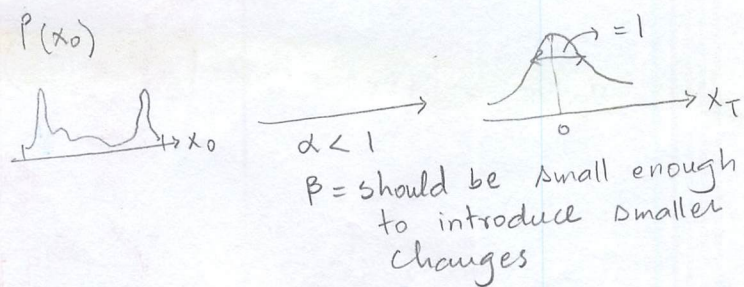
$$\Rightarrow X_t = \sqrt{1-\beta_t} X_{t-1} + X_{t-1} - X_{t-1} + \sqrt{\beta_t} \epsilon \quad \text{with } \epsilon \sim \mathcal{N}(0, I)$$

$X_t = \underbrace{X_{t-1}}_{\text{Previous State}} - \underbrace{(1 - \sqrt{1-\beta_t}) X_{t-1}}_{\text{Deterministic Term}} + \underbrace{\sqrt{\beta_t} \epsilon}_{\text{Stochastic Term}}$	Diffusion
New State Previous State Deterministic Term Stochastic Term	

Do we achieve our goal? Let's rewrite $\textcircled{1}$ as:

$$X_t = \sqrt{\alpha} X_{t-1} + \sqrt{\beta} \epsilon$$

- (forget relationship bw α, β atm)
- ($\alpha=0, \beta=1$ will get us our goal but that is not useful so we want gradual change with slow progression to $\mathcal{N}(0, I)$)



$\textcircled{1}: X_t = \sqrt{1-\beta} X_{t-1} + \sqrt{\beta} \mathcal{N}(0, I) \quad (\because \text{Assume } \beta_t = \beta \text{ for simplicity})$

$$\Rightarrow X_T = \sqrt{1-\beta} X_{T-1} + \sqrt{\beta} \mathcal{N}(\cdot)$$

$$X_T = \sqrt{1-\beta} \left(\sqrt{1-\beta} X_{T-2} + \sqrt{\beta} \mathcal{N}(\cdot) \right) + \sqrt{\beta} \mathcal{N}(\cdot)$$

$$X_T = (\sqrt{1-\beta})^2 X_{T-2} + (\sqrt{1-\beta}) (\sqrt{\beta}) \mathcal{N}(\cdot) + \sqrt{\beta} \mathcal{N}(\cdot)$$

$\vdots$

$$X_T = \underbrace{(\sqrt{1-\beta})^T}_{=0 \text{ as } T \rightarrow \infty} X_0 + \dots + \underbrace{\sqrt{1-\beta} \sqrt{1-\beta} \sqrt{\beta} \mathcal{N}(\cdot)}_{\text{Var} = \beta(1-\beta)^2} + \underbrace{\sqrt{1-\beta} \sqrt{\beta} \mathcal{N}(\cdot)}_{\text{Var} = \beta(1-\beta)} + \underbrace{\sqrt{\beta} \mathcal{N}(\cdot)}_{\text{Var} = \beta}$$

$$X_T = \underbrace{(\sqrt{1-\beta})^T X_0}_{=0} + \underbrace{\mathcal{N}(0, \beta(1-\beta)^T)}_{\downarrow} + \dots + \underbrace{\mathcal{N}(0, \beta I)}_{\downarrow}$$

$= \mathcal{N}(0, \sigma_T^2 I) = \mathcal{N}(0, I)$

Geometric series with:
1st term = β
factor = $1-\beta$

$$\sigma_T^2 = \beta \frac{1 - (1-\beta)^T}{1 - (1-\beta)} \approx \frac{\beta}{\beta} = 1$$

Sum of individual variances

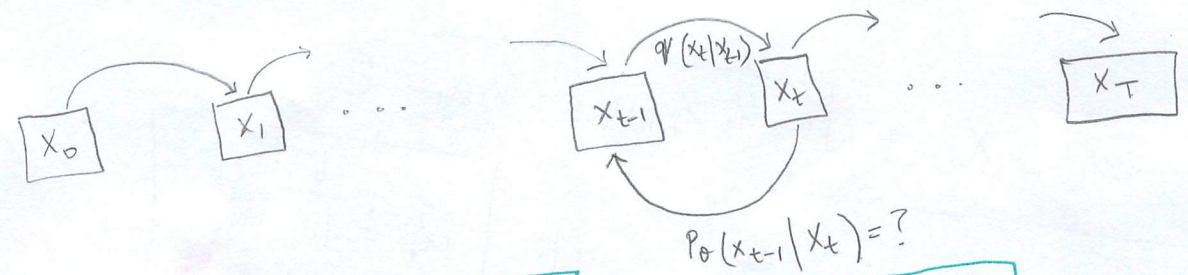
For $X_t = \sqrt{\alpha} X_{t-1} + \sqrt{\beta} \mathcal{N}(0, I)$, similar derivations reveal:

$$X_T = (\sqrt{\alpha})^T X_0 + \dots + \underbrace{\sqrt{\alpha} \sqrt{\alpha} \sqrt{\beta} \mathcal{N}(0, I)}_{\text{Var.} = \alpha^2 \beta} + \underbrace{\sqrt{\alpha} \sqrt{\beta} \mathcal{N}(0, I)}_{\text{Var.} = \alpha \beta} + \underbrace{\sqrt{\beta} \mathcal{N}(0, I)}_{\text{Var.} = \beta}$$

only = 0 if $\alpha < 1$

$$\sigma_T^2 = \beta \frac{1}{1-\alpha} \Rightarrow \beta = 1-\alpha \text{ for } \sigma_T^2 = 1$$

$\Rightarrow \alpha = 1-\beta$



Diffusion process that transforms any distribution to $\mathcal{N}(0, I)$

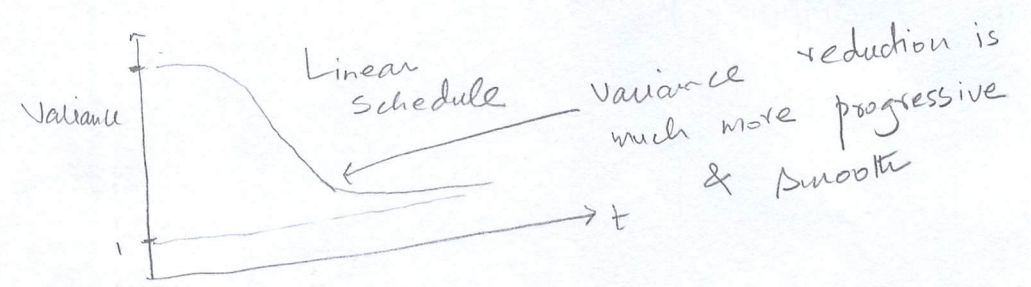
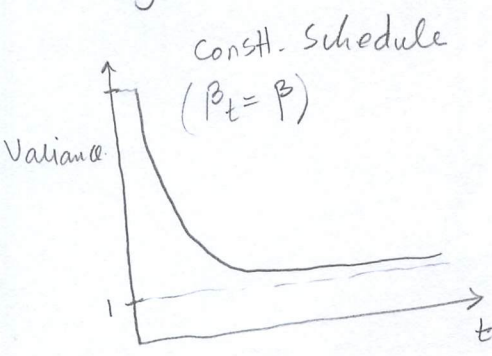
$$q(x_t | x_{t-1}) = \mathcal{N}(\sqrt{1-\beta_t} x_{t-1}, \beta_t I)$$

$$x_t = x_{t-1} \sqrt{1-\beta_t} + \sqrt{\beta_t} \mathcal{N}(0, I)$$

we use a scheduler on β as time progresses: e.g.
 $\beta_1 = 0.001 \rightarrow \beta_T = 0.02$

β_t -scheduling allows changing variance from input-variance x_0 to target (=1) x_T

Smoothly:



Can we go directly from x_0 to any x_t directly?

$$X_t = \sqrt{1-\beta_t} X_{t-1} + \sqrt{\beta_t} \mathcal{N}("") \quad \text{Let } \alpha_t = 1 - \beta_t$$

$$X_t = \sqrt{\alpha_t} X_{t-1} + \underbrace{\sqrt{1-\alpha_t} \mathcal{N}("")}_{\varepsilon_t}$$

$$X_t = \sqrt{\alpha_t} \left(\sqrt{\alpha_{t-1}} X_{t-2} + \sqrt{1-\alpha_{t-1}} \varepsilon_{t-1} \right) + \sqrt{1-\alpha_t} \varepsilon_t$$

$$X_t = \sqrt{\alpha_t} \sqrt{\alpha_{t-1}} X_{t-2} + \underbrace{\sqrt{\alpha_t} \sqrt{1-\alpha_{t-1}} \varepsilon_{t-1}}_{\text{var.} = \alpha_t (1-\alpha_{t-1})} + \underbrace{\sqrt{1-\alpha_t} \varepsilon_t}_{\text{var.} = 1-\alpha_t}$$

$$X_t = \sqrt{\alpha_t \alpha_{t-1}} X_{t-2} + \sqrt{1-\alpha_t + \alpha_t (1-\alpha_{t-1})} \varepsilon$$

$$X_t = \sqrt{\alpha_t \alpha_{t-1}} X_{t-2} + \sqrt{1-\alpha_t \alpha_{t-1}} \varepsilon$$

⋮

$$X_t = \sqrt{\alpha_t \alpha_{t-1} \alpha_{t-2}} X_{t-3} + \sqrt{1-\alpha_t \alpha_{t-1} \alpha_{t-2}} \varepsilon$$

⋮

$$X_t = \sqrt{\alpha_t \alpha_{t-1} \dots \alpha_2 \alpha_1} X_0 + \sqrt{1-\alpha_t \alpha_{t-1} \dots \alpha_2 \alpha_1} \varepsilon$$

$$X_t = \sqrt{\bar{\alpha}_t} X_0 + \sqrt{1-\bar{\alpha}_t} \varepsilon$$

with

$$\bar{\alpha}_t = \prod_{i=1}^t \alpha_i$$

Diagram showing the relationship between x_0 and x_t via the equation $x_t = \sqrt{\alpha_t} x_0 + \sqrt{1-\alpha_t} \varepsilon$.

Modeling forward-process like diffusion-process, the reverse process is also a diffusion-process with same functional form.